BIM Data Tools

What the Scripts Do and How to Use Them

This document explains two Python scripts developed for extracting and analysing data from BIM models. It is written for project managers and clients - no technical background is needed to follow it.

1. Script 1 — ifc_to_schedules.py

1.1 Purpose

This script reads one or more BIM model files (in the standard IFC format) and automatically produces the same spreadsheets you would normally export by hand from Revit.

Instead of opening each model in Revit and exporting each schedule individually, this script processes everything in one run and saves the results as CSV files that open directly in Excel.

1.2 What It Produces

For each model file, the script produces the following schedule types:

- Door Schedule
- Window Schedule
- Room / Space Schedule
- Wall Schedule and Wall Material Takeoff
- Floor Schedule and Floor Material Takeoff
- Ceiling Schedule
- Roof Schedule
- Structural Column Schedule
- Structural Framing Schedule
- Structural Foundation Schedule
- Curtain Panel and Mullion Schedules
- Duct Schedule
- Pipe Schedule
- Mechanical Equipment Schedule

It also produces a discovery report for each model, listing every element type found in the file and flagging anything not yet handled. A master spreadsheet merges all schedules across all projects.

1.3 Folder Structure

Place IFC files in the input folder before running the script. Outputs are written automatically to the output folder.

```
input_ifc\
    office_block.ifc
    apartment.ifc
```

```
output_csv\
    office_block_Door_Schedule.csv
    office_block_Window_Schedule.csv
    office_block_master.csv
    master_all_projects.csv
```

1.4 How to Run

1. Create a folder called input_ifc in the same location as the script.
2. Place your IFC model files inside it. Multiple files can be processed at once.
3. Open a terminal or command prompt, navigate to the script folder, and run:

```
python ifc_to_schedules.py
```

4. When the script finishes, open the output_csv folder to find all schedule files.

The script prints progress to the screen as it runs, including row counts per schedule and a summary of element types found in each file.

1.5 Requirements

Package	Install Command
Python 3.8 or later	https://python.org
ifcopenshell	pip install ifcopenshell
pandas	pip install pandas

2. Script 2 — analysis.py

2.1 Purpose

This script reads the master spreadsheet produced by Script 1 and runs three types of statistical analysis on the data. It is designed to surface patterns, inconsistencies, and relationships across BIM data that would be difficult to identify manually.

2.2 Analysis Methods

Clustering — Finding Natural Size Groups

The script groups elements by their dimensions using a clustering algorithm. It automatically identifies the best number of groups and reports the centre dimensions of each cluster. This is useful for:

- Identifying how many distinct door or window sizes are used across a project
- Spotting whether element types are consistent or contain many one-off sizes
- Flagging candidates for standardisation before procurement

Outlier Detection — Identifying Unusual Elements

The script uses two statistical methods (IQR and Z-score) to flag elements whose dimensions fall unusually far outside the normal range. Elements flagged by both methods are the highest-confidence findings. These may indicate:

- A modelling error — wrong units or a data entry mistake
- A non-standard specification requiring sign-off
- A genuine exception that should be documented

Correlation — Finding Linked Dimensions

The script checks whether different measurements tend to move together across elements and projects. Strong correlations are useful for:

- Understanding which dimensions are effectively linked in the model
- Informing parametric design rules or specification templates
- Cross-checking quantities during cost planning

2.3 Output Files

File	Contents
output_csv/analysis_findings_summary.csv	All findings across all three analyses
output_csv/outlier_results.csv	Every flagged outlier with value, average, and direction
output_csv/clustering_results.csv	Cluster centres and element counts
output_charts/*.png	All charts as images, ready to insert into reports

2.4 How to Run

5. Run ifc_to_schedules.py first to generate master_all_projects.csv.
6. Place master_all_projects.csv in the same folder as analysis.py.
7. Open a terminal, navigate to the folder, and run:

```
python analysis.py
```

8. When the script finishes, review output_charts/ for charts and output_csv/ for data tables.

2.5 Requirements

Package	Install Command
Python 3.8 or later	https://python.org
All analysis libraries	pip install pandas numpy matplotlib seaborn scipy scikit-learn

3. Troubleshooting

3.1 Script 1 — ifc_to_schedules.py

No output files produced

Check	Solution
input_ifc folder missing	Create the folder in the same directory as the script
No .ifc files in folder	Check files are present and named with the .ifc extension
ifcopenshell not installed	Run: pip install ifcopenshell

Schedule is empty (zero rows)

Check	Solution
IFC file has no elements of that type	Check the discovery report CSV for what the file actually contains
Export settings in Revit were restricted	Re-export from Revit with all categories enabled

3.2 Script 2 — analysis.py

Script fails to start

Check	Solution
master_all_projects.csv not found	Run ifc_to_schedules.py first and check the file was created
Missing Python libraries	Run: pip install pandas numpy matplotlib seaborn scipy scikit-learn

No charts produced

Check	Solution
Insufficient numeric data	The schedules need at least 5 numeric values per column to run analysis
output_charts folder not created	Check the terminal for error messages and re-run the script

4. Version History

Version	Date	Changes
1.0	April 2026	Initial release — ifc_to_schedules.py and analysis.py